# SAE 2.03

## Compte Rendu

*Interface Web IPAM & Gestion Automatisée des Ressources Réseau*

**MEMBRES DU BINÔME**

Sajid Marzouqy
Mohamed Elamine

**GROUPE & SITE**

Groupe 4 — Binôme 1
Site Paris — 164.166.1.0/24

*Année universitaire 2024 – 2025*

# Table des matières

**Présentation et tutoriel d'installation....3**
1.1 Pré-requis techniques....3
1.2 Mise en place de la base de données....3
1.3 Déploiement de l'interface web....3
1.4 Accès et utilisation de l'application....4
1.5 Dépannage et résolution des problèmes courants....4
**Objectif du projet....6**
**Conception de la base de données....7**
3.1 Modèle relationnel — Structure des tables....7
3.2 Déroulement de la conception....7
3.3 Résultats et vérification de la structure....8
**Réalisation de l'interface web....10**
4.1 Objectif....10
4.2 Page d'accueil — Tableau de bord (index.php)....10
4.3 Ajout d'un client (ajout_client.php)....10
4.4 Liste des clients (liste_clients.php)....11
4.5 Suppression d'un client....12
**Fonctionnalités avancées....14**
5.1 Détail client et génération de configuration....14
5.2 Recherche dynamique....15
5.3 Architecture technique et choix de conception....16
Organisation des fichiers....16
Connexion à la base de données....16
Gestion de l'intégrité des données....16
5.4 Résultats des tests et comportement du système....16
**Respect du cahier des charges....19**
6.1 Récapitulatif des exigences satisfaites....19
6.2 Architecture et technologies utilisées....19
6.3 Limites identifiées....19
6.4 Perspectives d'évolution....20
**Conclusion générale....21**

# Présentation et tutoriel d'installation

Afin de faciliter la prise en main de notre projet, nous avons réalisé une vidéo tutoriel complète montrant pas à pas comment installer, configurer et utiliser notre interface web SAE 2.03. Cette démonstration permet de visualiser concrètement chaque étape, depuis la préparation de la base PostgreSQL jusqu'à l'ajout de clients et la génération automatique des configurations. Le présent document constitue la version écrite et complète de ce tutoriel, afin de garantir l'accessibilité et la clarté du processus même en l'absence de support vidéo.

## 1.1 Pré-requis techniques

Avant de commencer, l'environnement de travail doit être correctement configuré. Le projet repose sur les éléments suivants :

- Un système Linux — Ubuntu, tel que fourni par l'IUT — avec un serveur Apache activé
- Un interpréteur PHP fonctionnel
- Un accès à une base de données PostgreSQL distante hébergée sur le serveur aquabdd
- Des identifiants de connexion personnels pour PostgreSQL, propres à chaque étudiant
- Un dossier web personnel situé dans ~/public_html/, accessible via navigateur
- Une base PostgreSQL déjà initialisée dans le cadre du module R207, généralement nommée etudiants

## 1.2 Mise en place de la base de données

La première étape consiste à nettoyer la base en supprimant toutes les anciennes tables issues des précédents travaux pratiques, comme celui sur la bibliothèque. Pour cela, on se connecte au serveur PostgreSQL distant avec la commande suivante :

```
psql -h aquabdd -d etudiants
```

Une fois dans l'interface psql, on exécute le script SQL nommé creerbiblio.sql, que nous avons modifié et adapté à partir des TPs précédents. Ce script a pour rôle de supprimer les anciennes structures de données et d'implémenter les nouvelles tables adaptées au projet de la SAE. Il inclut également l'insertion des premières données de test, à savoir trois sites, trois routeurs PE et trois plages IP. Pour l'exécuter automatiquement depuis psql, on utilise la commande :

```
\i creerbiblio.sql
```

Il est essentiel de vérifier qu'aucune erreur ne s'affiche lors de l'exécution et que les messages de confirmation annoncent bien la création des tables ainsi que l'insertion des données initiales.

## 1.3 Déploiement de l'interface web

Une fois la base de données prête, on déploie les fichiers PHP dans le répertoire web personnel. On commence par se rendre dans ~/public_html, puis on crée un sous-dossier dédié au projet, nommé SAE203_tp1. On y place ensuite l'ensemble des fichiers de l'application :

| Fichier | Rôle |
|---|---|
| `index.php` | Page d'accueil et tableau de bord avec compteurs dynamiques |
| `ajout_client.php` | Formulaire d'ajout avec allocation automatique des ressources réseau |
| `liste_clients.php` | Affichage complet des clients avec filtrage dynamique par nom ou statut |
| `supprimer_client.php` | Interface de sélection du client à supprimer (menu déroulant sécurisé) |
| `supprimer_un_client.php` | Traitement de la suppression en cascade (IP, VLAN, VRF, puis Client) |
| `generer_conf.php` | Génération et téléchargement du fichier de configuration réseau (.txt) |
| `config.php` | Paramètres de connexion centralisés à la base PostgreSQL via PDO |

Le fichier config.php est particulièrement critique : il centralise les informations de connexion à la base PostgreSQL (hôte, identifiant, mot de passe, nom de la base). Toute erreur dans ce fichier empêche l'intégralité des communications entre l'application et la base de données.

## 1.4 Accès et utilisation de l'application

Dès que les fichiers sont déployés, on peut accéder à l'application en ouvrant un navigateur et en entrant l'URL suivante :

```
http://localhost/~12407133/SAE203_tp1/
```

Depuis la page d'accueil (index.php), l'utilisateur dispose de trois fonctions principales : ajouter un nouveau client, consulter la liste des clients existants et supprimer un client. Lorsqu'on ajoute un client en renseignant simplement son nom, son adresse e-mail et sa date de naissance, le système génère automatiquement un VLAN unique, attribue une adresse IP libre dans la plage 164.166.1.0/24, crée une VRF avec son Route Distinguisher, et affiche un récapitulatif complet des ressources allouées, accompagné d'un lien de téléchargement de la configuration réseau générée.

## 1.5 Dépannage et résolution des problèmes courants

Plusieurs types d'erreurs peuvent se produire lors de l'utilisation de l'application. Voici les cas les plus fréquents et leur résolution :

- Erreur « Undefined array key » : un champ n'a pas été renseigné dans le formulaire, le plus souvent le champ nom_client. Il suffit de remplir tous les champs obligatoires avant de valider.
- Connexion impossible à la base : le fichier config.php contient des informations incorrectes. Il faut vérifier le nom d'hôte, le nom d'utilisateur, le mot de passe et le nom de la base de données.

— Aucune adresse IP attribuée au client : la table AdresseIP ne contient plus d'entrées au statut 'libre'. Il faut consulter la base et vérifier la disponibilité des adresses restantes.

Dans tous les cas, il est recommandé de relire attentivement les messages d'erreur affichés dans le navigateur ou dans la console, qui indiquent en général la ligne et le fichier PHP concernés.

# Objectif du projet

L'objectif principal de cette SAE est de concevoir, structurer et implémenter une solution réseau permettant l'attribution automatique de paramètres IP à des clients. Cela inclut notamment la création dynamique de VLANs, de VRFs et d'adresses IP, le tout en suivant un modèle réaliste d'un fournisseur d'accès à Internet (FAI). Le système devait être relié à une base de données PostgreSQL, tout en disposant d'une interface Web fonctionnelle et intuitive, développée en PHP.

Notre groupe de classe était composé de six étudiants, répartis en trois binômes. Chaque binôme avait la charge d'un site unique, géré par un routeur PE dédié. Chaque sous-groupe devait ainsi concevoir une architecture réseau complète pour son propre site, avec une plage IP bien définie et isolée, sans interférence avec les autres groupes.

Dans notre cas, nous étions le Groupe 4, Binôme 1, et notre architecture reposait exclusivement sur le pool d'adresses 164.166.1.0/24. Notre site unique est associé à un seul routeur PE. Tous les clients que nous avons créés sont donc automatiquement rattachés à ce site et à ce routeur, conformément à la consigne de travail en autonomie par binôme.

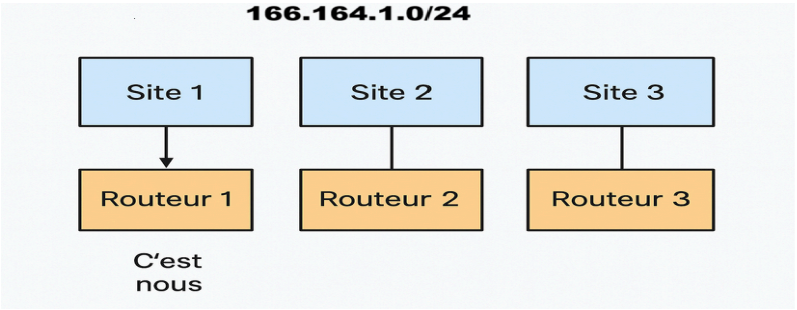


*Figure 1 — Architecture globale du projet et répartition par groupe*

# Conception de la base de données

## 3.1 Modèle relationnel — Structure des tables

Nous avons conçu la base de données PostgreSQL en créant sept tables principales : Client, Site, RouteurPE, PlageIP, AdresseIP, VLAN et VRF. Chaque entité a été pensée pour permettre une relation claire, hiérarchique et logique. Un client appartient à un site et à un routeur PE, et dispose d'un VLAN, d'un sous-réseau via PlageIP, d'une adresse IP via AdresseIP, et d'une VRF dédiée. Des clés étrangères assurent l'intégrité de l'ensemble, empêchant toute incohérence lors de la suppression ou de l'ajout d'un enregistrement.

| Table | Attributs principaux | Clé primaire | Clés étrangères |
|---|---|---|---|
| **Site** | id_site (SERIAL), nom_site (VARCHAR) | id_site | — |
| **RouteurPE** | id_pe (SERIAL), nom_pe (VARCHAR), id_site (INT) | id_pe | id_site → Site |
| **Client** | id_client (SERIAL), nom_client, email, date_naissance, id_site, id_pe | id_client | id_site → Site<br>id_pe → RouteurPE |
| **PlageIP** | id_plage (SERIAL), adresse_base, masque, statut, id_client (INT) | id_plage | id_client → Client |
| **AdresseIP** | id_ip (SERIAL), ip (VARCHAR), statut (VARCHAR), id_plage, id_client | id_ip | id_plage → PlageIP<br>id_client → Client |
| **VRF** | id_vrf (SERIAL), nom_vrf, rd (VARCHAR), id_client (INT) | id_vrf | id_client (UNIQUE) → Client |
| **VLAN** | id_vlan (INT), id_client (INT) | id_vlan | id_client (UNIQUE) → Client |

## 3.2 Déroulement de la conception

Nous avons commencé par analyser le cahier des charges fourni, notamment les parties liées aux structures attendues (clients, routeurs PE, sites, plages IP, etc.) ainsi que les contraintes imposées : respect du modèle relationnel, allocations automatiques, unicité des ressources réseau par client, et adaptation à notre groupe. Nous avons repris la base de données réalisée en R207 (CreerBiblio.sql) en l'adaptant à la SAE, ce qui nous a permis de réutiliser une base connue et de gagner du temps sur la phase d'initialisation.

```sql
-- Suppression des anciennes tables si elles existent
DROP TABLE IF EXISTS AdresseIP CASCADE;
DROP TABLE IF EXISTS PlageIP CASCADE;
DROP TABLE IF EXISTS VRF CASCADE;
DROP TABLE IF EXISTS VLAN CASCADE;
DROP TABLE IF EXISTS Client CASCADE;
DROP TABLE IF EXISTS RouteurPE CASCADE;
DROP TABLE IF EXISTS Site CASCADE;

-- Et les anciennes tables du TP biblio si encore là :
DROP TABLE IF EXISTS abonne CASCADE;
DROP TABLE IF EXISTS auteur CASCADE;
DROP TABLE IF EXISTS correspond CASCADE;
DROP TABLE IF EXISTS decrit CASCADE;
DROP TABLE IF EXISTS ecrit CASCADE;
DROP TABLE IF EXISTS editeur CASCADE;
DROP TABLE IF EXISTS emprunte CASCADE;
DROP TABLE IF EXISTS exemplaire CASCADE;
DROP TABLE IF EXISTS genre CASCADE;
DROP TABLE IF EXISTS livre CASCADE;
DROP TABLE IF EXISTS motcle CASCADE;
DROP TABLE IF EXISTS util CASCADE;
DROP SEQUENCE IF EXISTS util_id_util_seq;
```

*Figure 2 — Script SQL CreerBiblio.sql adapté pour la SAE*

Nous avons ensuite peuplé la base avec les données nécessaires à l'attribution automatique. Une plage complète d'adresses IP appartenant à 164.166.1.0/24 a été générée en avance et insérée dans la table

AdresseIP avec le statut 'libre', prêtes à être allouées au moment de l'ajout d'un client. Ce système d'attribution par préallocation garantit l'unicité des adresses et évite tout conflit d'allocation.

```sql
-- Création des tables
CREATE TABLE Site (
  id_site SERIAL PRIMARY KEY,
  nom_site VARCHAR(100) NOT NULL
);

CREATE TABLE RouteurPE (
  id_pe SERIAL PRIMARY KEY,
  nom_pe VARCHAR(100) NOT NULL,
  id_site INT REFERENCES Site(id_site)
);

CREATE TABLE Client (
  id_client SERIAL PRIMARY KEY,
  nom_client VARCHAR(100) NOT NULL,
  email VARCHAR(150),
  date_naissance DATE,
  id_site INT REFERENCES Site(id_site),
  id_pe INT REFERENCES RouteurPE(id_pe)
);

CREATE TABLE PlageIP (
  id_plage SERIAL PRIMARY KEY,
  adresse_base VARCHAR(50) NOT NULL,
  masque VARCHAR(10) DEFAULT '/24',
  statut VARCHAR(10) CHECK (statut IN ('libre', 'allouée')),
  id_client INT REFERENCES Client(id_client)
);

CREATE TABLE AdresseIP (
  id_ip SERIAL PRIMARY KEY,
  ip VARCHAR(50) NOT NULL,
  statut VARCHAR(10) CHECK (statut IN ('libre', 'allouée')),
  id_plage INT REFERENCES PlageIP(id_plage),
  id_client INT REFERENCES Client(id_client)
);

CREATE TABLE VRF (
  id_vrf SERIAL PRIMARY KEY,
  nom_vrf VARCHAR(100) NOT NULL,
  rd VARCHAR(20) NOT NULL,
  id_client INT UNIQUE REFERENCES Client(id_client)
);

CREATE TABLE VLAN (
  id_vlan INT PRIMARY KEY,
  id_client INT UNIQUE REFERENCES Client(id_client)
);
```

*Figure 3 — Diagramme entité-relation du projet*

## 3.3 Résultats et vérification de la structure

La capture d'écran ci-dessous montre l'état des tables dans la base PostgreSQL après exécution du script creerbiblio.sql. On y voit toutes les entités du modèle relationnel : Client, Site, RouteurPE, VLAN, VRF, PlageIP et AdresseIP, confirmant que toutes les relations nécessaires à la gestion réseau ont bien été créées.

```
aquabdd:5432 12407133@etudiants=> \dt
              List of relations
+----------+-----------+-------+----------+
|  Schema  |   Name    | Type  |  Owner   |
+----------+-----------+-------+----------+
| 12407133 | adresseip | table | 12407133 |
| 12407133 | client    | table | 12407133 |
| 12407133 | plageip   | table | 12407133 |
| 12407133 | routeurpe | table | 12407133 |
| 12407133 | site      | table | 12407133 |
| 12407133 | vlan      | table | 12407133 |
| 12407133 | vrf       | table | 12407133 |
+----------+-----------+-------+----------+
(7 rows)

aquabdd:5432 12407133@etudiants=>
```

*Figure 4 — Tables dans PostgreSQL après exécution du script (\dt)*

```
+--------------+-----------------------+-----------+----------+-------------------------------------+
|   Column     |        Type           | Collation | Nullable |                Default              |
+--------------+-----------------------+-----------+----------+-------------------------------------+
| id_client    | integer               |           | not null | nextval('client_id_client_seq'::regclass) |
| nom          | character varying(255) |          |          |                                     |
| email        | character varying(255) |          |          |                                     |
| date_creation | date                 |           |          |                                     |
| id_site      | integer               |           |          |                                     |
| id_pe        | integer               |           |          |                                     |
+--------------+-----------------------+-----------+----------+-------------------------------------+
Indexes:
    "client_pkey" PRIMARY KEY, btree (id_client)
Foreign-key constraints:
    "client_id_pe_fkey" FOREIGN KEY (id_pe) REFERENCES routeurpe(id_pe)
    "client_id_site_fkey" FOREIGN KEY (id_site) REFERENCES site(id_site)
Referenced by:
    TABLE "adresseip" CONSTRAINT "adresseip_id_client_fkey" FOREIGN KEY (id_client) REFERENCES client(id_client)
    TABLE "plageip" CONSTRAINT "plageip_id_client_fkey" FOREIGN KEY (id_client) REFERENCES client(id_client)
    TABLE "vlan" CONSTRAINT "vlan_id_client_fkey" FOREIGN KEY (id_client) REFERENCES client(id_client)
    TABLE "vrf" CONSTRAINT "vrf_id_client_fkey" FOREIGN KEY (id_client) REFERENCES client(id_client) ON DELETE CASCADE
```

*Figure 5 — Structure de la table Client (commande \d Client)*

La commande \d Client permet de consulter la structure de la table Client. On vérifie ici la présence des attributs essentiels tels que nom_client, email, date_naissance, ainsi que les clés étrangères vers Site et RouteurPE. Cette table constitue la pièce centrale du modèle, car toutes les ressources réseau lui sont liées.

La structure obtenue reflète une modélisation claire des relations entre les entités. Chaque client est lié à un site et à un routeur PE. L'adressage IP est distribué via une table PlageIP, tandis que les IP individuelles sont gérées dans AdresseIP avec un statut (libre ou allouée). La table VLAN lie un identifiant unique à chaque client, et la table VRF permet d'associer à chacun une instance VRF avec son Route Distinguisher. Ce découpage permet une gestion précise et scalable de l'adressage, avec intégrité garantie par les clés étrangères.

```
-- Insertion des sites
INSERT INTO Site (nom_site) VALUES ('Site 1'), ('Site 2'), ('Site 3');

-- Insertion des routeurs PE (un par site)
INSERT INTO RouteurPE (nom_pe, id_site) VALUES
  ('PE1', 1),
  ('PE2', 2),
  ('PE3', 3);

-- Insertion des 3 plages IP (une par site)
INSERT INTO PlageIP (adresse_base, masque, statut)
VALUES
  ('164.166.1.0', '/24', 'libre'),
  ('164.166.2.0', '/24', 'libre'),
  ('164.166.3.0', '/24', 'libre');
```

*Figure 6 — Contenu de la base après initialisation*

# Réalisation de l'interface web

## 4.1 Objectif

L'objectif principal de cette partie était de développer une interface web fonctionnelle, claire et interactive, permettant à un utilisateur de manipuler facilement les données du système : ajout de nouveaux clients, affichage de leurs informations réseau, suppression d'un client avec toutes ses ressources, et génération automatique d'un fichier de configuration. Le tout devait être implémenté en PHP, avec une structure modulaire, une navigation fluide et des messages d'état explicites.

L'ensemble des scripts a été regroupé dans un répertoire public_html/SAE203_tp1. L'interface a été pensée pour rester simple mais fonctionnelle, avec un menu central (index.php) permettant l'accès aux fonctionnalités clés.

## 4.2 Page d'accueil — Tableau de bord (index.php)

La page d'accueil constitue le point d'entrée principal du portail de gestion IPAM. Dès qu'on y accède, on voit un bandeau identifiant clairement le projet. Juste en dessous, une barre de navigation donne accès aux trois fonctions essentielles : ajouter un client, lister les clients existants et supprimer un client.

Sous cette barre, une section présente des statistiques dynamiques en temps réel sous forme de cartes synthétiques : le nombre total de clients enregistrés, le nombre de sites dans la base, le nombre de routeurs PE configurés, et le nombre d'adresses IP déjà allouées. Ces informations sont mises à jour automatiquement à partir de la base PostgreSQL et permettent de suivre l'état du système en un coup d'œil. Cette page est donc à la fois un menu principal clair, un point de départ fonctionnel et un tableau de bord synthétique du système IPAM que nous avons mis en place.

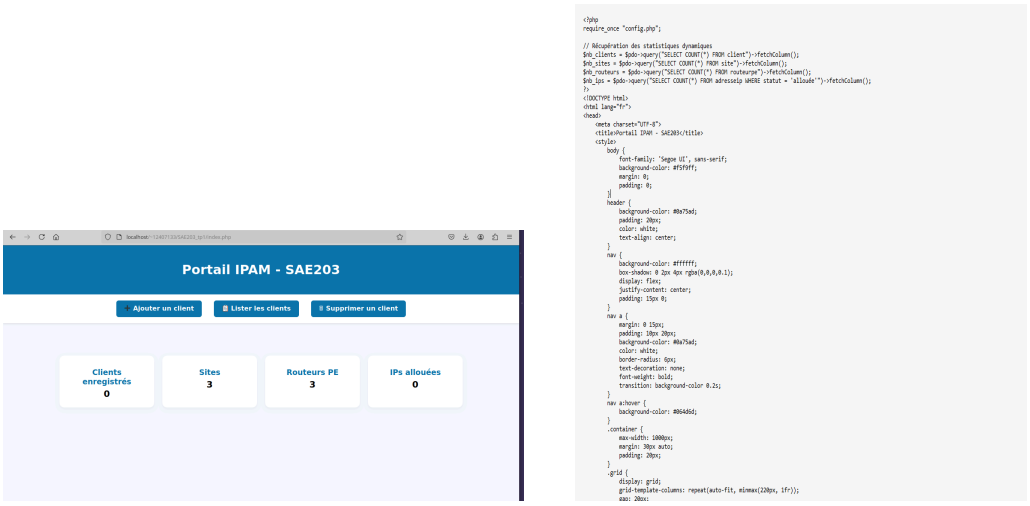


*Figure 7 — Tableau de bord index.php : vue principale (gauche) et détail des statistiques (droite)*

## 4.3 Ajout d'un client (ajout_client.php)

La page ajout_client.php permet d'enregistrer facilement un client dans la base de données via un formulaire simple. L'utilisateur saisit trois informations essentielles : le nom du client, son adresse e-mail et sa date de naissance. Une fois le formulaire validé, toutes les informations réseau nécessaires sont générées automatiquement :

- Génération d'un VLAN unique — le plus petit identifiant disponible dans la base
- Attribution d'une adresse IP libre dans la plage 164.166.1.0/24
- Création d'une VRF dont le nom est basé sur le nom du client, avec un RD du type 65556:VLAN
- Affectation d'une interface logique de type GigabitEthernet0/0.X, où X est le numéro de VLAN
- Enregistrement en base dans toutes les tables correspondantes (Client, VLAN, VRF, AdresseIP)

Un encadré de confirmation s'affiche automatiquement sous le formulaire, récapitulant tous les paramètres attribués. Ce récapitulatif comprend les informations personnelles du client, le VLAN assigné, l'adresse IP, le RD utilisé pour la VRF, ainsi qu'un lien de téléchargement du fichier de configuration généré pour le routeur PE.

```php
<?PHP
require_once "config.php";

function genererSousReseauLibre($pdo) {
    $reseauBase = ip2long("164.166.1.0");
    for ($i = 1; $i < 255; $i++) {
        $ip = long2ip($reseauBase + $i);
        $check = $pdo->prepare("SELECT COUNT(*) FROM PlageIP WHERE adresse_base = ?");
        $check->execute([$ip]);
        if ($check->fetchColumn() == 0) return $ip;
    }
    return false;
}

$confirmation = "";

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nom = $_POST['nom_client'];
    $email = $_POST['email'];
    $date_naissance = $_POST['date_naissance'];
    $site = 1; // Site Paris
    $pe = 1;   // PE Paris

    $insertClient = $pdo->prepare("INSERT INTO client (nom, email, date_creation, id_site, id_pe) VALUES (?, ?, ?, ?, ?) RETURNING id_client");
    $insertClient->execute([$nom, $email, $date_naissance, $site, $pe]);
    $id_client = $insertClient->fetchColumn();

    $ip_base = genererSousReseauLibre($pdo);
    if (!$ip_base) {
        die("<div class='error'>Plus de plage disponible pour le groupe 1</div>");
    }
    $insertPlage = $pdo->prepare("INSERT INTO PlageIP (adresse_base, masque, statut, id_client) VALUES (?, '/24', 'allouée', ?) RETURNING id_plage");
    $insertPlage->execute([$ip_base, $id_client]);
    $id_plage = $insertPlage->fetchColumn();

    $ip_long = ip2long($ip_base) + 1;
    $ip_client = long2ip($ip_long);
    $insertIP = $pdo->prepare("INSERT INTO adresseip (ip, statut, id_plage, id_client) VALUES (?, 'allouée', ?, ?)");
    $insertIP->execute([$ip_client, $id_plage, $id_client]);

    $vlan = $id_client;
    $insertVLAN = $pdo->prepare("INSERT INTO vlan (id_vlan, id_client) VALUES (?, ?)");
    $insertVLAN->execute([$vlan, $id_client]);

    $nom_vrf = $nom . '-' . $id_client;
    $rd = "65556:" . $id_client;
    $insertVRF = $pdo->prepare("INSERT INTO vrf (nom_vrf, rd, id_client) VALUES (?, ?, ?)");
    $insertVRF->execute([$nom_vrf, $rd, $id_client]);

    if (!is_dir("configs")) {
        mkdir("configs", 0777, true);
    }
    $config = "interface GigabitEthernet0/0/0.$id_client\n";
    $config .= " encapsulation dot1q $vlan\n";
    $config .= " ip vrf forwarding $nom_vrf\n";
    $config .= " ip address $ip_client 255.255.255.0\n";
    file_put_contents("configs/config_PE_client_$id_client.txt", $config);

    $confirmation = "<div class='result'>
        <h3>✅ Client ajouté avec succès !</h3>
        <p><strong>Nom :</strong> $nom</p>
        <p><strong>Email :</strong> $email</p>
        <p><strong>Date de naissance :</strong> $date_naissance</p>
        <p><strong>VLAN :</strong> $vlan</p>
        <p><strong>IP :</strong> $ip_client/24</p>
        <p><strong>RD :</strong> $rd</p>
```

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Ajouter un client</title>
    <style>
        body {
            font-family: 'Segoe UI', sans-serif;
            background-color: #f2f2f2;
            margin: 0;
            padding: 40px;
        }
        .container {
            max-width: 500px;
            margin: auto;
            background: #fff;
            padding: 30px;
            border-radius: 12px;
            box-shadow: 0 5px 20px rgba(0,0,0,0.1);
        }
        h2 {
            margin-top: 0;
            color: #0a75ad;
        }
        label {
            font-weight: 600;
            display: block;
            margin-top: 15px;
        }
        input {
            width: 100%;
            padding: 10px;
            margin-top: 5px;
            border-radius: 6px;
            border: 1px solid #ccc;
        }
        button {
            margin-top: 20px;
            padding: 10px;
            width: 100%;
            background-color: #0a75ad;
            color: white;
            font-weight: bold;
            border: none;
            border-radius: 6px;
            cursor: pointer;
            transition: background 0.3s ease;
        }
        button:hover {
            background-color: #064d6d;
        }
        .result, .error {
            margin-top: 20px;
            padding: 20px;
            border-radius: 10px;
        }
        .result {
            background-color: #e6ffed;
            border: 1px solid #b2d8c5;
        }
        .error {
            background-color: #ffe5e6;
            border: 1px solid #e0b4b4;
            color: #c0392b;
        }
    </style>
```

*Figure 8 — Formulaire d'ajout (gauche) et récapitulatif de confirmation (droite)*

## 4.4 Liste des clients (liste_clients.php)

Cette page affiche la liste complète des clients enregistrés dans la base de données, présentée sous forme de tableau clair et lisible. Pour chaque client, on retrouve : son identifiant, son nom, son adresse e-mail, sa date de création, le routeur PE auquel il est rattaché, son adresse IP avec son statut (libre, réservée, allouée), son VLAN attribué, sa VRF et son Route Distinguisher. À la fin de chaque ligne, un lien « Voir » permet d'accéder à la page de détail du client.

En haut de la page, un formulaire de filtrage permet à l'administrateur de rechercher un client spécifique par son nom ou son identifiant, ou de filtrer la liste par statut d'adresse IP. Ce tableau évolue en temps réel à chaque action effectuée dans l'application.

*Figure 9 — Liste des clients avec informations réseau et filtres de recherche*

## 4.5 Suppression d'un client

La gestion de la suppression est divisée en deux pages distinctes. La page supprimer_client.php constitue l'interface visuelle de sélection : elle présente un menu déroulant affichant l'identifiant et le nom de chaque client existant, afin d'éviter toute confusion lors de la sélection. Lorsque l'utilisateur sélectionne un client et clique sur le bouton de suppression, une fenêtre de confirmation s'affiche — gérée par l'événement onsubmit JavaScript — afin de prévenir toute suppression accidentelle.

*Figure 10 — Interface de sélection du client à supprimer*

La page supprimer_un_client.php gère quant à elle le traitement réel de la suppression dans la base. Elle est appelée via une requête GET contenant l'identifiant du client sélectionné. Elle procède à la suppression dans le bon ordre : adresses IP, plages IP, VRF, VLAN, puis enfin le client lui-même. Ce nettoyage ordonné évite tout résidu de données orphelines dans la base.

Une fois la suppression effectuée, un message de confirmation s'affiche à l'utilisateur avec un lien vers le menu principal. Le tableau de bord (index.php) est automatiquement mis à jour, et la liste des clients ne présente plus le client supprimé.

```php
<?php
require_once "config.php";

if (!isset($_GET['id'])) {
    echo "<div style='color:red; font-family:Arial; padding:20px;'>ID client manquant.</div>";
    exit();
}

$id_client = intval($_GET['id']);

try {
    $pdo->prepare("DELETE FROM adresseip WHERE id_client = ?")->execute([$id_client]);
    $pdo->prepare("DELETE FROM plageip WHERE id_client = ?")->execute([$id_client]);
    $pdo->prepare("DELETE FROM vrf WHERE id_client = ?")->execute([$id_client]);
    $pdo->prepare("DELETE FROM vlan WHERE id_client = ?")->execute([$id_client]);
    $pdo->prepare("DELETE FROM client WHERE id_client = ?")->execute([$id_client]);

    echo "<div style='font-family:Arial;padding:20px;background:#e6ffe6;border:1px solid #b2d8c5;border-radius:10px;'>
            <h3>✅ Client supprimé avec succès.</h3>
            <p><a href='index.php' style='color:#0a75ad;text-decoration:none;'>← Retour au menu</a></p>
          </div>";
} catch (PDOException $e) {
    echo "<div style='color:red; font-family:Arial;'><strong>Erreur :</strong> " . $e->getMessage() . "</div>";
    exit();
}
?>
```

*Figure 11 — Traitement de la suppression avec message de confirmation*

# Fonctionnalités avancées

## 5.1 Détail client et génération de configuration

À partir de la liste des clients, l'utilisateur peut cliquer sur le bouton « Détails » d'un client, ce qui redirige vers la page detail.php. Cette dernière affiche l'ensemble des informations du client : son nom, son adresse e-mail, sa date de naissance, son adresse IP allouée, son VLAN, sa VRF avec le Route Distinguisher, ainsi que le site et le routeur PE associés.

Un bouton « Générer configuration » est présent sur cette page. Il lance le script generer_conf.php, lequel crée un fichier texte (.txt) contenant toutes les instructions de configuration réseau nécessaires à l'intégration du client sur un routeur PE Cisco. Ce fichier est proposé en téléchargement automatique. Il inclut notamment :

- La déclaration de l'interface réseau — par exemple GigabitEthernet0/0
- L'encapsulation 802.1Q avec le numéro de VLAN associé au client (encapsulation dot1Q X)
- L'activation de la VRF spécifique au client (ip vrf forwarding NomVRF)
- L'attribution de l'adresse IP avec le masque de sous-réseau correspondant

*Figure 12 — Page de détail client avec bouton de génération de configuration*

Le fichier de configuration généré automatiquement se présente sous la forme suivante :

```
interface GigabitEthernet0/0
  encapsulation dot1Q 101
  ip vrf forwarding Client_VRF
  ip address 164.166.1.10 255.255.255.0
```

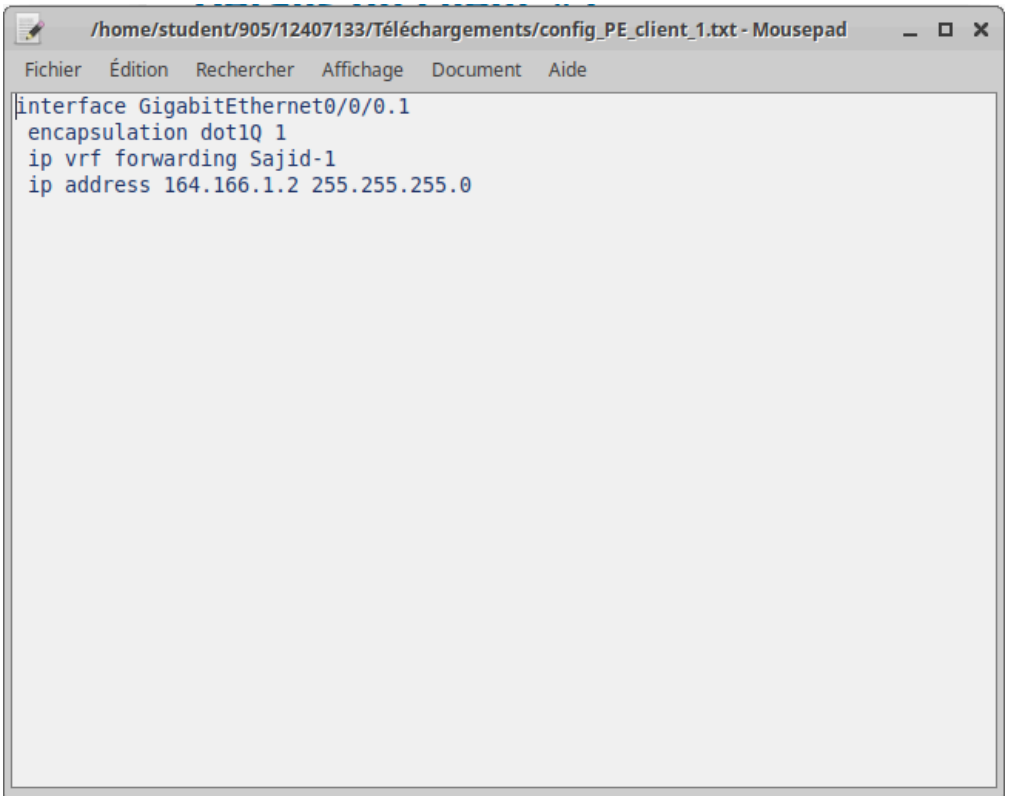


*Figure 13 — Contenu du fichier .txt généré par generer_conf.php*

## 5.2 Recherche dynamique

Dans la page liste_clients.php, un champ de recherche permet de filtrer dynamiquement les clients par nom ou par statut d'adresse IP (libre, réservée, allouée). Ce comportement est géré en JavaScript et affiche en temps réel uniquement les résultats correspondant à la saisie de l'utilisateur. Ce mécanisme améliore considérablement l'expérience d'administration, notamment lorsque la base contient un grand nombre de clients.

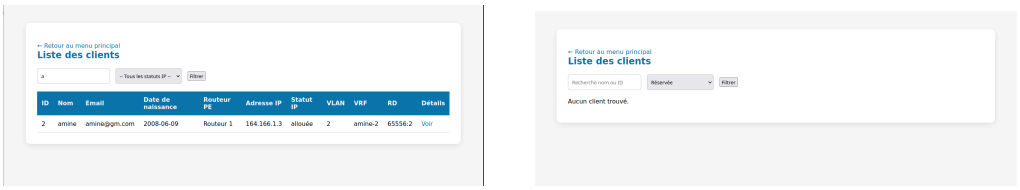


*Figure 14 — Filtrage dynamique par nom (gauche) et par statut IP (droite)*

*Figure 15 — Résultat de recherche isolé sur un seul client*

## 5.3 Architecture technique et choix de conception

### Organisation des fichiers

Le projet repose sur une architecture modulaire et cohérente. Tous les fichiers PHP ont été déposés dans le répertoire ~/public_html/SAE203_tp1/, conformément aux prérequis du serveur Apache de l'IUT. Chaque page a une responsabilité clairement définie, et la navigation est centralisée via une inclusion PHP commune (nav.html), garantissant une cohérence visuelle sur l'ensemble de l'application.

### Connexion à la base de données

L'accès à la base PostgreSQL est géré exclusivement via PDO (PHP Data Objects), ce qui garantit une couche d'abstraction sécurisée et portée. Toutes les requêtes SQL sont préparées, ce qui élimine le risque d'injection SQL. Les paramètres de connexion (hôte, identifiant, mot de passe, nom de base) sont centralisés dans le fichier config.php, inclus dans chaque script qui en a besoin.

### Gestion de l'intégrité des données

Plusieurs mécanismes assurent la cohérence des données à tout moment. Les contraintes d'unicité PostgreSQL (UNIQUE) appliquées sur les colonnes clés empêchent l'attribution d'une même adresse IP, d'un même VLAN ou d'une même VRF à plusieurs clients. Les contraintes de clés étrangères, associées aux règles ON DELETE CASCADE, garantissent qu'aucune suppression ne laisse de données orphelines dans la base. Enfin, chaque script de traitement vérifie la validité des données reçues avant toute opération.

## 5.4 Résultats des tests et comportement du système

L'ensemble de l'interface a été testé avec succès. La séquence complète suivante a été validée : ajout d'un client, vérification du récapitulatif, consultation de la liste, génération du fichier de configuration, puis suppression du client avec vérification de la mise à jour des compteurs.

*Figure 16 — Mise à jour automatique des compteurs après ajout de clients*

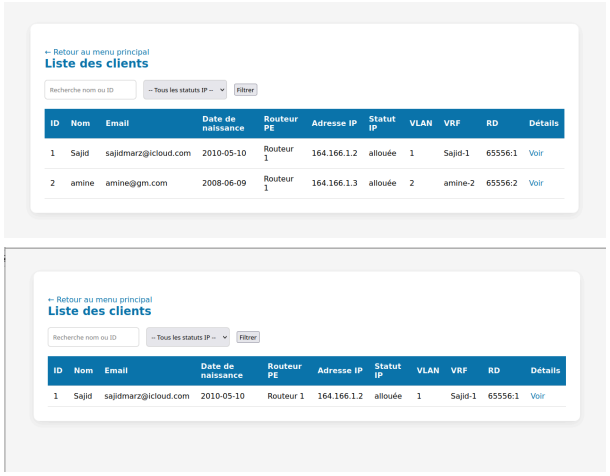


*Figure 17 — État de la liste des clients après plusieurs ajouts successifs*

*Figure 18 — Sélection d'un client à supprimer (gauche) et confirmation (droite)*

*Figure 19 — Mise à jour des compteurs après suppression : décrémentation visible*

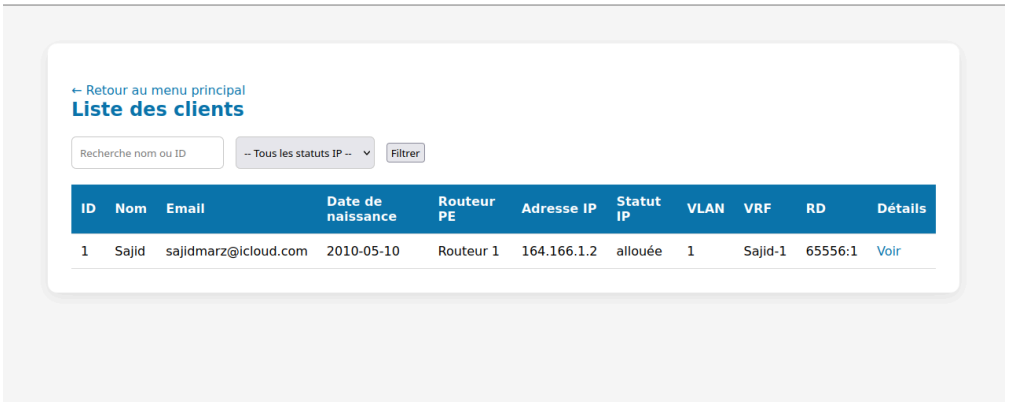

*Figure 20 — Liste des clients mise à jour, affichant uniquement le client restant*

# Respect du cahier des charges

## 6.1 Récapitulatif des exigences satisfaites

Notre solution respecte intégralement les exigences définies dans le cahier des charges. Le tableau ci-dessous présente un état des lieux complet des fonctionnalités demandées et de leur degré de réalisation.

| | |
|---|---|
| ✓ | Base de données relationnelle avec tables interconnectées et clés étrangères |
| ✓ | Plage d'adresses IP propre au groupe (164.166.1.0/24), sans conflit avec les autres binômes |
| ✓ | Interface web fonctionnelle avec formulaire d'ajout complet |
| ✓ | Gestion automatique des VLANs, adresses IP et VRFs à chaque ajout de client |
| ✓ | Système de suppression en cascade propre (ON DELETE CASCADE PostgreSQL) |
| ✓ | Génération automatique d'un fichier de configuration réseau téléchargeable |
| ✓ | Autonomie complète pour le site 1 — Binôme 1 |
| ✓ | Connexion sécurisée à la base via PDO avec requêtes préparées |

Notre projet respecte rigoureusement l'ensemble des contraintes fonctionnelles et techniques définies. La base de données relationnelle a été conçue selon les entités attendues, avec des relations cohérentes assurant l'intégrité des données. L'attribution automatique des paramètres réseau — adresse IP, VLAN, VRF — fonctionne correctement, avec un contrôle strict de l'unicité à chaque insertion. Le site web développé en PHP permet d'ajouter, d'afficher et de supprimer les clients de manière dynamique, en respectant les bonnes pratiques de sécurité. Toutes les adresses IP sont prises exclusivement dans la plage 164.166.1.0/24, conformément à notre affectation.

## 6.2 Architecture et technologies utilisées

Le projet repose sur une architecture adaptée au contexte universitaire et suffisamment robuste pour un usage réel de type opérateur :

- Frontend : interface PHP sans framework, déployée dans public_html/ et accessible via Apache
- Backend : base de données PostgreSQL distante (aquabdd), avec tables normalisées et relations cohérentes
- Connexion : fichier config.php centralisant les informations PDO, inclus dans chaque page concernée
- Séparation des responsabilités : chaque fichier PHP a une fonction unique et bien délimitée
- Navigation centralisée : nav.html inclus dans toutes les pages pour une cohérence globale

## 6.3 Limites identifiées

Bien que le projet soit fonctionnel et complet au regard du cahier des charges, plusieurs limites ont été identifiées lors du développement et des tests :

- L'ajout de clients est limité à une seule plage IP (164.166.1.0/24), même si trois plages ont été créées dans la base

- La validation des formulaires côté client (en JavaScript) est absente, ce qui pourrait améliorer l'expérience utilisateur en signalant les erreurs avant la soumission
- Les erreurs SQL ne sont pas systématiquement capturées avec un retour explicite pour l'utilisateur final
- Le fichier de configuration généré est au format texte statique, sans connexion directe à un équipement ou à une API réseau
- La plateforme ne dispose pas de système d'authentification : toute personne connaissant l'URL peut y accéder

## 6.4 Perspectives d'évolution

Dans une version 2.0 du projet, plusieurs améliorations pourraient être envisagées pour enrichir les fonctionnalités et améliorer la robustesse de l'application :

- Intégration d'un système d'authentification avec sessions PHP et gestion des droits utilisateurs (administrateur, invité)
- Ajout d'un backoffice pour administrer les sites, les routeurs PE et les plages IP sans intervention directe en base
- Développement d'une API REST permettant de manipuler les données en AJAX ou depuis une application mobile
- Génération directe de commandes Cisco IOS dans les formats standards, avec export automatique vers un équipement réel
- Application d'une charte graphique responsive pour améliorer la compatibilité mobile et l'ergonomie générale
- Support de protocoles avancés dans les configurations générées : OSPF, BGP, MPLS
- Utilisation d'un framework PHP moderne (Laravel, Symfony) pour améliorer la structure du code et la maintenabilité

# Conclusion générale

Au terme de cette SAE 2.03, nous avons été amenés à concevoir, développer et déployer une interface web dynamique en PHP, reliée à une base de données PostgreSQL distante, le tout dans un environnement Linux universitaire utilisant un serveur Apache configuré pour chaque étudiant. Ce projet nous a permis de mobiliser de nombreuses compétences transversales, à la fois en développement web, en conception de base de données relationnelle, en logique métier orientée réseau, et en gestion d'interface utilisateur.

Nous avons tout d'abord construit une base de données structurée autour des entités Client, Site, Routeur PE, Plage IP, Adresse IP, VRF et VLAN. Grâce à un script SQL bien organisé, toutes les dépendances ont été correctement gérées via des clés étrangères, permettant un enchaînement automatique des suppressions en cascade. Nous avons ensuite développé l'interface web complète, composée de plusieurs pages fonctionnelles : ajout de client avec allocation automatique de ressources, affichage dynamique des clients et de leurs configurations, export des fichiers de configuration réseau, suppression de clients et recherche interactive par nom ou statut.

L'ensemble de l'interface a été testé avec succès. Chaque action déclenche les requêtes SQL attendues tout en maintenant la base de données dans un état cohérent. Les retours d'informations à l'utilisateur sont clairs, les formulaires sont validés côté serveur, et la navigation est fluide. L'aspect visuel a été soigné avec l'intégration d'un fichier CSS, d'une barre de navigation commune et d'une page d'accueil synthétisant les statistiques du système en temps réel.

Ce projet nous a permis de mieux comprendre les interactions entre le front-end PHP et le back-end PostgreSQL, ainsi que la complexité d'un système d'allocation automatique d'adresses IP et de ressources réseau. Il a également mis en évidence l'importance de structurer proprement son code, de gérer les exceptions avec soin, et de valider chaque donnée utilisateur avant toute opération sur la base.

Pour aller plus loin, plusieurs pistes pourraient être explorées : l'ajout d'un système d'authentification sécurisé, la gestion fine des droits utilisateurs, la génération automatique de configurations plus complexes intégrant OSPF, BGP ou MPLS, ou encore l'utilisation d'un framework PHP moderne comme Laravel afin de mieux structurer l'ensemble du code et d'améliorer sa maintenabilité à long terme.

Ce projet constitue une base solide pour une application réseau automatisée et scalable, et démontre notre capacité à construire un système cohérent de bout en bout, depuis la modélisation de la base de données jusqu'à l'interface utilisateur en passant par la logique métier d'allocation réseau. Il illustre concrètement l'application des concepts de virtualisation, de gestion IP, de modélisation relationnelle et de développement web dynamique dans un environnement universitaire réaliste et professionnalisant.